# Signed p-adic Residual Encodings of Finite-Domain All-Different Systems with a Sudoku Case Study

Greg Baker
School of Computing, Australian National University
Canberra, Australia
greg.baker@anu.edu.au
ORCID: 0000-0002-6910-3011

**Abstract**

We study signed, weighted affine $p$-adic residual objectives as native encodings of finite-domain constraints. For primes that separate the finite alphabet, sufficiently weighted positive unary rows pin each coefficient to its allowed set, while negative rows reward unequal endpoints or clause satisfaction. A coordinatewise domination theorem places every global minimiser in the finite domain; there the loss is, up to an additive constant, the all-different conflict count or the negative number of satisfied CNF clauses. Standard Sudoku provides an 81-coefficient case study without a one-hot lift. A client-side implementation exposes the generated dataframes, arithmetic, diagnostics, and searches.

**Keywords:** p-adic regression; signed residual objectives; all-different constraints; list colouring; Sudoku

## 1 Introduction

In affine $p$-adic linear regression, an observation pairs a feature vector $u \in \mathbb{Z}^n$ with a target $b \in \mathbb{Z}$, and a coefficient vector $x$ is scored by a weighted sum of residual norms $|u^\top x - b|_p$. Earlier work establishes the geometry and complexity of this problem when every weight is positive [1, 2, 16]. This paper studies the signed variant, in which an observation may carry a negative weight. In ordinary least squares, negative weights can destroy convexity and boundedness. Over $\mathbb{Z}_p$, by contrast, every affine residual has norm at most one, so a finite signed objective remains bounded and a negative row acts as a bounded reward.

This makes $p$-adic linear regression surprisingly expressive. Finite-domain all-different systems and CNF formulae compile mechanically into signed objectives: sufficiently weighted positive unary rows pin each coefficient to a finite allowed set, and a coordinatewise domination theorem (Theorem 1) shows that every global minimiser lies in that finite domain. There each remaining residual norm is a zero-or-one indicator, so a negative row rewards an unequal pair $x_i \neq x_j$ or a satisfied clause, and the loss is, up to an additive constant, the conflict count of the all-different system or the negative number of satisfied clauses. The global minimisers are exactly the domain-respecting minimum-conflict assignments, and exactly the satisfying assignments when the instance is satisfiable (Theorem 3, Corollary 4).

This expressivity also gives a short proof that signed $p$-adic linear regression is NP-hard: the clause compiler reduces 3-SAT to the threshold problem at the fixed prime $p = 5$ (Corollary 5). Earlier NP-hardness results treated an ordinary summed 2-adic objective [2], while Mihara's forcing theorem proves the positive-weight problem NP-hard for every fixed prime [16].

The paper makes the following contributions.

- A compiler template from finite-domain all-different systems and CNF formulae to signed weighted affine $p$-adic residual objectives, with a coordinatewise domination theorem locating every global minimiser in the finite domain, on $\mathbb{Z}_p^n$ and on $\mathbb{Q}_p^n$ (Theorem 1, Corollary 2).
- An exact characterisation of the global minimisers for all-different systems in list-colouring form as the domain-respecting minimum-conflict assignments, covering the unsatisfiable Max-CSP regime and the antiferromagnetic Potts reading (Theorem 3, Section 4).
- NP-hardness of the signed threshold problem at the fixed prime $p = 5$, by an elementary reduction through the clause compiler (Corollary 5).
- Polynomial-size positive-only complement expansions for the negative CNF and pairwise all-different rows, supporting a diagnostic comparison with Mihara's digitwise equality regression while the signed rows are retained for valuation and loss reporting (Appendix A).
- A Sudoku case study on the native 81-cell digit space using a variety of $p$-adic linear-regression techniques; a negative result for a powers-of-two encoding; and a client-side browser implementation exposing the generated dataframes, loss arithmetic, and searches (Sections 5–7, Appendices B–D).

Section 2 fixes notation, works a three-variable list-colouring example and a two-variable CNF example in full, and records a family of integer false labellings with the uniform labelling as its degenerate member and a member whose residual values identify the satisfied literals; a polynomial-valued variant is deferred to Appendix G. Section 3 states and proves the general template and its corollaries. Section 4 treats unsatisfiable instances. Sections 5 and 6 specialise the construction to Sudoku. Section 7 reports the computations, while Section 8 locates the construction among valued CSPs and Potts energies. The appendices compare Mihara's digitwise regression, record the heuristics and their traces, report the experiments and failed powers-of-two encoding, state the label and coefficient requirements together with an approximate-negation variant, and give a polynomial-valued labelling.

## 2 $p$-adic linear regression

Fix a prime $p$. For a nonzero integer $n$, let $v_p(n)$ be the exponent of $p$ in its factorisation and define

$$|n|_p = p^{-v_p(n)}, \qquad |0|_p = 0.$$

This norm makes numbers close when their difference is divisible by a large power of $p$. Ordinary (Euclidean) weighted least squares normally uses nonnegative weights. Inserting negative coefficients into its quadratic objective can destroy convexity and boundedness, but this does not happen for the $p$-adic objective used here. In affine $p$-adic linear regression, an observation with feature vector $u$ and target $b$ has residual $u^\top x - b$, and the usual positive-weight objective sums terms of the form

$$\gamma\, |u^\top x - b|_p, \qquad \gamma > 0.$$

This is a weighted $p$-adic linear-regression objective. Earlier results establish geometric and complexity properties of this positive-weight problem [1, 2]; broader recent work develops classification, regression, and representation-learning primitives over the $p$-adics [14]. We find no prior treatment of the signed affine objective used below. On $\mathbb{Z}_p$, every affine residual in this paper has norm at most one, so each finite signed objective is bounded despite its negative terms. We demonstrate that such objectives give useful representations of constraint-satisfaction problems: source forms can be transformed mechanically into datasets, and a global minimiser can be transformed mechanically into a satisfying CSP assignment when one exists, or otherwise into a domain-respecting minimum-conflict assignment.

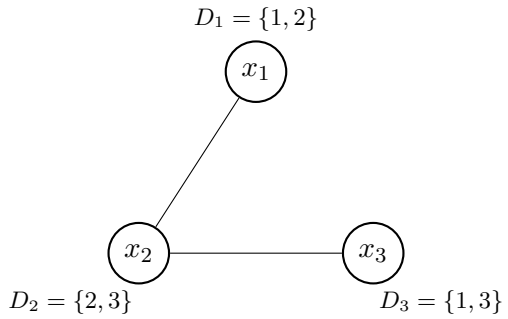


Figure 1: A tiny list-colouring instance. The assignment $x_1 = 1$, $x_2 = 2$, $x_3 = 3$ respects the domains and makes every edge unequal.

| Row | $x_1$ | $x_2$ | $x_3$ | Target | Signed weight | Role |
|---|---|---|---|---|---|---|
| $P_1$ | 1 | 0 | 0 | 1 | +3 | domain well |
| $P_2$ | 1 | 0 | 0 | 2 | +3 | domain well |
| $P_3$ | 0 | 1 | 0 | 2 | +3 | domain well |
| $P_4$ | 0 | 1 | 0 | 3 | +3 | domain well |
| $P_5$ | 0 | 0 | 1 | 1 | +3 | domain well |
| $P_6$ | 0 | 0 | 1 | 3 | +3 | domain well |
| $E_1$ | 1 | −1 | 0 | 0 | −1 | reward $x_1 \neq x_2$ |
| $E_2$ | 0 | 1 | −1 | 0 | −1 | reward $x_2 \neq x_3$ |

Table 1: Dataframe view of Figure 2. The three coefficient columns form the feature vector, and each row contributes its signed weight times the 5-adic norm of feature dot coefficient minus target.

### 2.1 An example

We begin with an example rather than the general theorem. Figure 1 has three variables. Each variable must take a value from its displayed domain, and adjacent variables must be different. The assignment $x = (1, 2, 3)$ is one solution.

For this example take $p = 5$. The only fact about the 5-adic norm needed at first is that $|0|_5 = 0$, while every nonzero integer residual that occurs below has norm 1. A dataframe row consists of a feature vector $u$, target $b$, and signed weight $w$; at coefficient vector $x$ it contributes

$$w\,|u^\top x - b|_5$$

to the objective. Positive rows of weight 3 pin each coefficient to its allowed values. Negative rows of weight $-1$ reward unequal endpoints.

The eight observations have two equivalent presentations: Table 1 and Figure 2 which uses a compact tuple notation, in which each tuple $(\sigma, u, b, \gamma)$ has signed weight $w = \sigma\gamma$.

The zero-intercept model is the hyperplane

$$b = u^\top x = u_1x_1 + u_2x_2 + u_3x_3.$$

The general theorem below (Theorem 1 on page 6) guarantees that at least one global minimum occurs among the finitely many domain-respecting candidates.

On a domain-respecting vector the six positive rows contribute 9, while each unequal edge subtracts one. Thus

$$L_{\text{toy}}(x) = 9 - \mathbf{1}_{x_1 \neq x_2} - \mathbf{1}_{x_2 \neq x_3}.$$

Brute-force evaluation gives Table 2.

The minimum-loss coefficient vectors are exactly the valid assignments of Figure 1; in particular, $x = (1, 2, 3)$ is recovered as a regression optimum.

| Row | Type | Synthetic observation | Role |
|---|---|---|---|
| $P_1$ | positive | $(+1, e_1, 1, 3)$ | pin $x_1$ to $D_1$ |
| $P_2$ | positive | $(+1, e_1, 2, 3)$ | pin $x_1$ to $D_1$ |
| $P_3$ | positive | $(+1, e_2, 2, 3)$ | pin $x_2$ to $D_2$ |
| $P_4$ | positive | $(+1, e_2, 3, 3)$ | pin $x_2$ to $D_2$ |
| $P_5$ | positive | $(+1, e_3, 1, 3)$ | pin $x_3$ to $D_3$ |
| $P_6$ | positive | $(+1, e_3, 3, 3)$ | pin $x_3$ to $D_3$ |
| $E_1$ | negative | $(-1, e_1 - e_2, 0, 1)$ | reward $x_1 \neq x_2$ |
| $E_2$ | negative | $(-1, e_2 - e_3, 0, 1)$ | reward $x_2 \neq x_3$ |

Figure 2: Synthetic dataset for the toy list-colouring instance in Figure 1. Here $e_1, e_2, e_3$ denote the standard basis vectors of $\mathbb{Z}^3$.

| Coefficient vector $x$ | $L_{\text{toy}}(x)$ | Proper list-colouring? |
|---|---|---|
| $(1, 2, 1)$ | 7 | yes |
| $(1, 2, 3)$ | 7 | yes |
| $(1, 3, 1)$ | 7 | yes |
| $(1, 3, 3)$ | 8 | no |
| $(2, 2, 1)$ | 8 | no |
| $(2, 2, 3)$ | 8 | no |
| $(2, 3, 1)$ | 7 | yes |
| $(2, 3, 3)$ | 8 | no |

Table 2: All eight hyperplanes determined by the domain-pinning rows.

### 2.2 $p$-adic linear regression can express CNF statements as well

Consider the CNF expression in the two Boolean variables $z_1$ and $z_2$:

$$\Phi = (z_1) \wedge (\neg z_1 \vee z_2).$$

What values of $z_1$ and $z_2$ make $\Phi$ true? Encode $x_i = 0$ as $z_i = \text{true}$ and $x_i = 1$ as $z_i = \text{false}$. A clause is false at one affine equality, so a negative row rewards every coefficient vector away from that equality. The first clause is false when $x_1 = 1$, giving the residual $x_1 - 1$. The second is false when $(x_1, x_2) = (0, 1)$, giving $-x_1 + x_2 - 1$. With $p = 5$ and Boolean-pinning weight 3, the complete dataframe is Table 3.

We want to make sure that $x_1$ and $x_2$ take values only in $\{0, 1\}$, so we make two deep holes ("Boolean wells") in the loss landscape for each variable. If the wells are deeper than the rewards from the clause-derived rows, then the optimal values of $x_1$ and $x_2$ are suitably constrained.

| Row | $x_1$ | $x_2$ | Target | Signed weight | Role |
|---|---|---|---|---|---|
| $C_1$ | 1 | 0 | 1 | $-1$ | reward $(z_1)$ |
| $C_2$ | $-1$ | 1 | 1 | $-1$ | reward $(\neg z_1 \vee z_2)$ |
| $U_{1,0}$ | 1 | 0 | 0 | $+3$ | Boolean well |
| $U_{1,1}$ | 1 | 0 | 1 | $+3$ | Boolean well |
| $U_{2,0}$ | 0 | 1 | 0 | $+3$ | Boolean well |
| $U_{2,1}$ | 0 | 1 | 1 | $+3$ | Boolean well |

Table 3: Signed regression dataframe compiled from the two-clause formula $\Phi$. Constraint rows reward nonzero residuals; well rows force Boolean coefficients.

Choosing one well for $x_1$ and one for $x_2$ enumerates the four zero-intercept hyperplanes through the corresponding two basis rows. On Boolean coefficients the wells contribute 6, so the loss is $6 - \text{sat}_\Phi(x)$:

| $x$ | $(z_1, z_2)$ | Clauses satisfied | Loss |
|---|---|---|---|
| $(0,0)$ | (true, true) | 2 | 4 |
| $(0,1)$ | (true, false) | 1 | 5 |
| $(1,0)$ | (false, true) | 1 | 5 |
| $(1,1)$ | (false, false) | 1 | 5 |

Table 4: Brute-force regression for $\Phi$.

The unique minimum has coefficients $x = (0,0)$, which decode to the satisfying assignment $z_1 = z_2 = \text{true}$.

### 2.3 General false labels

The preceding CNF uses 0 for true and 1 for false for both variables. The false values need not coincide. They could be other $p$-adic integers, and they need not be distinct.

Within $\mathbb{Z}_p$ itself, the signed clause construction works verbatim for any false labels $b_i \equiv 1 \pmod{p}$. Because integer norms are at most 1, the domination arguments of Section 3 are unaffected. Distinct labels add information rather than correctness: with $b_i = 1 + p\,2^i$, a subset of $k$ labels sums to $k + p\sum_i 2^i$, so distinct subsets have distinct sums—a cardinality collision would require $p$ to divide a nonzero difference of cardinalities smaller than $p$—and the residual value then identifies exactly which literals of a clause are satisfied: the count from the residue modulo $p$, the set from the binary expansion of the quotient. The companion browser's Boolean-CSP page implements both labellings: switching to $b_i = 1 + p\,2^i$ leaves every displayed loss unchanged, while its row-evaluation table decodes each clause residual into the satisfied literals it names. Appendix E states the general conditions under which labels and clause coefficients preserve the clause-count loss, while Appendix F notes that negation may be encoded with non-negative coefficients. Appendix G records a polynomial-valued variant of these labels as a direction for future research.

## 3 A general all-different residual-objective template

The worked examples can now be put on a general footing. We work in the $p$-adic integers $\mathbb{Z}_p$, where every norm is at most 1. If $p > q$, the difference between any two distinct values in $\{1, \ldots, q\}$ has norm 1. For comparison, the discrete metric $d_{\text{disc}}(s,t) := \mathbf{1}_{s \neq t}$ is also an ultrametric; a discrete version of the construction is stated after the main theorem.

Many finite-domain all-different systems can be viewed as list-colouring problems. Each variable $x_i$ has a finite allowed set $D_i$, and an edge $\{i,j\}$ requires $x_i \neq x_j$. An all-different constraint of arity $r$ contributes the $\binom{r}{2}$ pairwise edges of its primal-graph clique.

**Definition 1** (Signed synthetic $p$-adic residual dataset)**.** *For parameters $x \in \mathbb{Z}_p^n$, a signed weighted affine observation is a quadruple $(\sigma, u, b, \gamma)$, where $\sigma \in \{+1, -1\}$ is the sign, $u \in \mathbb{Z}^n$ is the feature vector, $b \in \mathbb{Z}$ is the target, and $\gamma > 0$ is the weight. It contributes the term*

$$\sigma\gamma\,|u^\top x - b|_p$$

*to the loss; this is the basic term in the signed weighted $p$-adic objective.*

Since $u \in \mathbb{Z}^n$, $b \in \mathbb{Z}$, and $x \in \mathbb{Z}_p^n$, each residual lies in $\mathbb{Z}_p$ and has norm at most 1. Every finite signed objective considered on this domain is therefore bounded. An observation is *positive* when $\sigma = +1$ and *negative* when $\sigma = -1$; positive observations will pin coefficients and negative observations will supply bounded rewards.

**Theorem 1** (Finite-domain signed affine compiler template)**.** *Fix a prime $p$. Let $D_i \subseteq \mathbb{Z}$ be finite nonempty sets whose elements are pairwise incongruent modulo $p$, and put*

$$D := \prod_{i=1}^{n} D_i \subseteq \mathbb{Z}_p^n.$$

*For each coordinate $i$, choose a pinning weight $\lambda_i > 0$. Let $\mathcal{T}$ be a finite family of signed affine observations*

$$(\sigma_\ell, u_\ell, b_\ell, \gamma_\ell), \qquad \sigma_\ell \in \{\pm 1\}, \quad u_\ell \in \mathbb{Z}^n, \quad b_\ell \in \mathbb{Z}, \quad \gamma_\ell > 0,$$

*and define*

$$L(x) := \sum_{i=1}^{n} \lambda_i \sum_{a \in D_i} |x_i - a|_p \; + \; \sum_{\ell \in \mathcal{T}} \sigma_\ell \gamma_\ell |u_\ell^\top x - b_\ell|_p.$$

*Assume that, for every coordinate $i$,*

$$\lambda_i > \sum_{\ell:\,(u_\ell)_i \neq 0} \gamma_\ell,$$

*and that, for every $x \in D$ and every $\ell \in \mathcal{T}$, the residual $u_\ell^\top x - b_\ell$ is either $0$ or a $p$-adic unit. Then $L$ attains its global minimum on $D$, and every global minimiser lies in $D$. On $D$, $L$ is a constant plus a finite signed objective whose interaction terms have values $0$ and $\sigma_\ell \gamma_\ell$.*

*Proof.* Suppose $x_i \notin D_i$. If $x_i$ is congruent modulo $p$ to an element of $D_i$, let $b$ be that element; otherwise choose any $b \in D_i$. Let $x'$ be obtained from $x$ by replacing $x_i$ with $b$.

The $i$-th unary sum decreases by at least $\lambda_i |x_i - b|_p$. Indeed, if $x_i \equiv b \pmod{p}$, then the only changed unary contribution below 1 is $|x_i - b|_p$. If $x_i$ is congruent to no element of $D_i$, then $|x_i - a|_p = 1$ for all $a \in D_i$, while the unary sum at $b$ has one zero term and all other terms equal to 1; in this case $|x_i - b|_p = 1$.

For an interaction term with $(u_\ell)_i \neq 0$, write $r'_\ell = u_\ell^\top x' - b_\ell$. Then

$$u_\ell^\top x - b_\ell = r'_\ell + (u_\ell)_i (x_i - b).$$

Since $(u_\ell)_i \in \mathbb{Z}$, $|(u_\ell)_i|_p \leq 1$, and the ultrametric inequality gives

$$\big| |u_\ell^\top x - b_\ell|_p - |r'_\ell|_p \big| \leq |x_i - b|_p.$$

Thus the total interaction increase caused by changing coordinate $i$ is at most

$$\sum_{\ell:\,(u_\ell)_i \neq 0} \gamma_\ell |x_i - b|_p.$$

By the assumed domination inequality, $L(x') < L(x)$. Repeating this coordinate-snapping step gives a point of $D$ with smaller loss than any starting point outside $D$. Since $D$ is finite, the global minimum is attained on $D$, and no global minimiser lies outside $D$.

On $D$, every unary sum is constant: exactly one term is zero and the rest are units. The second assumption says each interaction residual is either zero or a unit, so each signed interaction term is either 0 or $\sigma_\ell \gamma_\ell$. This proves the stated finite-domain form of $L|_D$. □

**Corollary 2** (Extension to $\mathbb{Q}_p^n$)**.** *Under the assumptions of Theorem 1, view the same formula for $L$ as an objective on $\mathbb{Q}_p^n$. It remains bounded below, attains its global minimum on $D$, and has no global minimiser outside $D$.*

*Proof.* Theorem 1 handles every coordinate in $\mathbb{Z}_p \setminus D_i$. If instead $|x_i|_p > 1$, then $|x_i - a|_p = |x_i|_p$ for every integer $a \in D_i$. Replacing $x_i$ by any $b \in D_i$ decreases its unary sum by

$$\lambda_i\big(|D_i|\,|x_i|_p - (|D_i| - 1)\big) \geq \lambda_i |x_i|_p = \lambda_i |x_i - b|_p.$$

The interaction increase is bounded by $\sum_{\ell:(u_\ell)_i \neq 0} \gamma_\ell |x_i - b|_p$, exactly as in the theorem. The domination inequality therefore makes the replacement strictly improving. Snapping all coordinates first into $\mathbb{Z}_p$ and then into $D$ proves the claim. In particular, every value of $L$ is bounded below by the finite minimum attained on $D$. □

**Definition 2** (Synthetic dataset for list-colouring)**.** *Let $V = \{1, \ldots, n\}$, let $G = (V, E)$ be a finite graph, let $D_i \subseteq \{1, \ldots, q\}$ be nonempty allowed sets, and fix a prime $p > q$. Define*

$$w_{ii} := 0, \qquad w_{ij} := \begin{cases} 1, & \{i,j\} \in E, \\ 0, & \{i,j\} \notin E, \end{cases} \qquad (i \neq j),$$

*let $\Delta := \max_{i \in V} \sum_{j=1}^{n} w_{ij}$, and set $\lambda := 1 + \Delta$. Writing $e_1, \ldots, e_n$ for the standard basis vectors of $\mathbb{Z}^n$, define*

$$\mathcal{S}^+ := \{(+1, e_i, a, \lambda) : i \in V,\ a \in D_i\},$$
$$\mathcal{S}^- := \{(-1, e_i - e_j, 0, 1) : 1 \leq i < j \leq n,\ w_{ij} = 1\}.$$

*Set $\mathcal{S} := \mathcal{S}^+ \sqcup \mathcal{S}^-$.*

The theorem is stated for a simple graph, so $w_{ij} \in \{0, 1\}$. An edge-weighted version is obtained by replacing the unit negative observation on edge $\{i, j\}$ with weight $c_{ij} > 0$ and choosing the pinning weight larger than $\max_i \sum_j c_{ij}$, but the binary case is the one needed here.

**Definition 3** (Loss associated with the list-colouring dataset)**.** *For $\mathcal{S}$ as in Definition 2, define*

$$L_{\mathcal{S}}(x) = \lambda \sum_{i=1}^{n} \sum_{a \in D_i} |x_i - a|_p - \sum_{1 \leq i < j \leq n} w_{ij} |x_i - x_j|_p.$$

This dataset has exactly $\sum_{i=1}^{n} |D_i|$ positive observations and $|E|$ negative observations, so it is computable in time $O(\sum_i |D_i| + |E|)$. The positive observations pin coordinates to allowed values, and the negative observations reward inequality across edges.

Each positive observation $(+1, e_i, a, \lambda)$ contributes $\lambda |x_i - a|_p$ to the loss, and each negative observation $(-1, e_i - e_j, 0, 1)$ contributes $-|x_i - x_j|_p$ to the loss. Summing them yields the displayed loss. The size bound and runtime are immediate from the explicit definitions.

The compiler template already implies that minimisers snap to the finite alphabet, because the pinning weight is larger than the degree of every vertex. The next proof records the stronger fact needed for all-different systems: after snapping, the objective is exactly a constant plus the edge-conflict count.

### 3.1 Correctness

Since $p > q$, the residues of $1, \ldots, q$ are distinct modulo $p$. In particular, for any fixed $i \in V$ and any $t \in \mathbb{Z}_p$, there is at most one $a \in D_i$ with $t \equiv a \pmod{p}$.

First we record the simple observation that, on $\{1, \ldots, q\}$, the $p$-adic norm detects inequality exactly.

**Lemma 1** (Edge terms become inequality indicators on the domain)**.** *If $x_i, x_j \in \{1, \ldots, q\}$ and $p > q$, then*

$$|x_i - x_j|_p = \begin{cases} 0, & x_i = x_j, \\ 1, & x_i \neq x_j. \end{cases}$$

*Proof.* If $x_i = x_j$ then the difference is zero. If $x_i \neq x_j$, then $1 \leq |x_i - x_j| \leq q - 1 < p$, so $p \nmid (x_i - x_j)$ and therefore $|x_i - x_j|_p = 1$. □

We next prove a digit-snapping lemma: if $x$ minimises $L_{\mathcal{S}}$, then each coordinate $x_i$ lies in $D_i$. The explicit unary-well formula makes that argument transparent.

**Lemma 2** (Unary wells)**.** *For $i \in V$, define*

$$U_i(t) := \sum_{a \in D_i} |t - a|_p \qquad (t \in \mathbb{Z}_p).$$

*Then $U_i(t) \geq |D_i| - 1$, with equality if and only if $t \in D_i$.*

*Proof.* If $t \in D_i$, then one summand in $U_i(t)$ is 0, and the remaining $|D_i| - 1$ summands are 1, so $U_i(t) = |D_i| - 1$.

If $t \notin D_i$ but $t \equiv b \pmod{p}$ for some $b \in D_i$, then $0 < |t-b|_p < 1$, while for every $a \in D_i \setminus \{b\}$ we have $t - a \equiv b - a \not\equiv 0 \pmod{p}$, so $|t - a|_p = 1$. Hence

$$U_i(t) = (|D_i| - 1) + |t - b|_p > |D_i| - 1.$$

If $t$ is congruent modulo $p$ to no element of $D_i$, then $|t - a|_p = 1$ for all $a \in D_i$, so $U_i(t) = |D_i| > |D_i| - 1$. □

**Lemma 3** (Digit-snapping lemma)**.** *Let $x \in \mathbb{Z}_p^n$. Suppose there exists $i \in V$ with $x_i \notin D_i$. Choose $b \in D_i$ as follows: if $x_i$ is congruent modulo $p$ to an element of $D_i$, take that unique element; otherwise choose any $b \in D_i$. Let $x'$ be obtained from $x$ by replacing $x_i$ with $b$. Then*

$$L_{\mathcal{S}}(x') < L_{\mathcal{S}}(x).$$

*Proof.* The positive part of the loss changes by

$$\lambda \left( \sum_{a \in D_i} |b - a|_p - \sum_{a \in D_i} |x_i - a|_p \right).$$

There are two cases to consider.

If $x_i$ is congruent modulo $p$ to no element of $D_i$, then $|x_i - a|_p = 1$ for every $a \in D_i$, so $\sum_{a \in D_i} |x_i - a|_p = |D_i|$. On the other hand, one summand in $\sum_{a \in D_i} |b - a|_p$ is 0 and the remaining $|D_i| - 1$ summands are 1, so $\sum_{a \in D_i} |b - a|_p = |D_i| - 1$. Hence the positive change is $-\lambda$. Since $|x_i - b|_p = 1$ in this case, this is at most $-\lambda |x_i - b|_p$.

If $x_i \equiv b \pmod{p}$ for some $b \in D_i$, then $\sum_{a \in D_i} |b - a|_p = |D_i| - 1$, whereas

$$\sum_{a \in D_i} |x_i - a|_p = (|D_i| - 1) + |x_i - b|_p.$$

Hence the positive change is $-\lambda |x_i - b|_p$.

For each $j \in V$, the corresponding negative component changes by

$$w_{ij} \big( |x_i - x_j|_p - |b - x_j|_p \big).$$

By the ultrametric inequality,

$$|x_i - x_j|_p \leq \max\big( |x_i - b|_p,\ |b - x_j|_p \big),$$

so

$$|x_i - x_j|_p - |b - x_j|_p \leq |x_i - b|_p.$$

Summing over all $j \in V$,

$$L_{\mathcal{S}}(x') - L_{\mathcal{S}}(x) \leq \left( -\lambda + \sum_{j=1}^{n} w_{ij} \right) |x_i - b|_p \leq -|x_i - b|_p < 0.$$

□

On domain-respecting assignments, the loss is an additive constant plus the conflict count.

**Theorem 3** (Correctness of the synthetic reduction)**.** *For the dataset $\mathcal{S} = \mathcal{S}^+ \sqcup \mathcal{S}^-$ from Definition 2 and the loss from Definition 3, every global minimiser of $L_{\mathcal{S}}$ lies in $\prod_{i=1}^n D_i$. On $\prod_{i=1}^n D_i$,*

$$L_{\mathcal{S}}(x) = \lambda \sum_{i=1}^{n} (|D_i| - 1) - |E| + \sum_{1 \leq i < j \leq n} w_{ij} \mathbf{1}_{x_i = x_j}.$$

*Consequently, the global minimisers are exactly the domain-respecting assignments that minimise the conflict sum $\sum_{1 \leq i < j \leq n} w_{ij} \mathbf{1}_{x_i = x_j}$, or equivalently maximise the number of unequal edges. In particular, if the list-colouring instance $(G, (D_i)_{i=1}^n)$ is satisfiable, then this minimum conflict sum is 0, and the global minimisers are exactly its proper list-colourings.*

*Proof.* By Lemma 3, any point with some coordinate outside its domain can be improved by snapping that coordinate into the domain. Therefore every global minimiser lies in $\prod_i D_i$. If $x \in \prod_i D_i$, then Lemma 2 gives $U_i(x_i) = |D_i| - 1$ for every $i$, and Lemma 1 gives

$$|x_i - x_j|_p = \mathbf{1}_{x_i \neq x_j} \qquad (1 \leq i < j \leq n).$$

Since $\mathbf{1}_{x_i \neq x_j} = 1 - \mathbf{1}_{x_i = x_j}$ on every edge, substituting these identities into $L_{\mathcal{S}}$ yields the displayed formula. A global minimiser exists: $L_{\mathcal{S}}$ takes only finitely many values on the finite set $\prod_i D_i$, and by the previous paragraph no point outside $\prod_i D_i$ can be a global minimiser, so the global minimum is attained on $\prod_i D_i$. There the displayed formula is an additive constant plus the conflict sum, so a domain-respecting assignment is a global minimiser if and only if it attains the least possible conflict sum; this is the stated "exactly" characterisation in both directions. In particular, if $(G, (D_i)_{i=1}^n)$ is satisfiable the least conflict sum is 0, and the global minimisers are exactly its proper list-colourings. □

**Corollary 4.** *Any finite-domain all-different constraint system can be encoded, in polynomial time, as a signed weighted synthetic p-adic residual objective whose global minimisers are exactly the domain-respecting assignments with minimum conflict count. If the constraint system is satisfiable, these minimisers are exactly its satisfying assignments.*

*Proof.* Relabel the union of the explicitly listed domain values by $\{1, \ldots, q\}$, preserving equality across domains. Here $q \leq \sum_i |D_i|$. Choose a prime $p > q$ (for $q > 1$, Bertrand's postulate gives one below $2q$); because $q$ is bounded by the explicit input length, scanning this interval with deterministic primality testing is polynomial in that input length. Form the primal graph: the vertices are the variables, and two variables are adjacent if they appear together in some all-different constraint. The satisfying assignments are exactly the proper list-colourings of this graph, with the variable domains as lists, and the minimum-conflict assignments are exactly the domain-respecting assignments minimising the corresponding deduplicated edge-conflict sum. Apply Definitions 2 and 3, and Theorem 3. □

**Discrete-metric variant.** The same synthetic dataset also works with the discrete metric

$$d_{\mathrm{disc}}(s,t) := \mathbf{1}_{s\neq t}.$$

If each residual $|u^\top x - b|_p$ is replaced by $d_{\mathrm{disc}}(u^\top x, b)$, then the loss becomes

$$L_{\mathcal{S}}^{\mathrm{disc}}(x) = \lambda \sum_{i=1}^{n} \sum_{a\in D_i} \mathbf{1}_{x_i\neq a} \; - \sum_{1\le i<j\le n} w_{ij}\mathbf{1}_{x_i\neq x_j}.$$

The unary wells satisfy

$$U_i^{\mathrm{disc}}(t) := \sum_{a\in D_i} \mathbf{1}_{t\neq a} = \begin{cases} |D_i|-1, & t\in D_i, \\ |D_i|, & t\notin D_i. \end{cases}$$

Hence replacing any $x_i \notin D_i$ by a value $b \in D_i$ decreases the positive part by exactly $\lambda$, while the total increase in the negative part is at most $\sum_j w_{ij} \le \Delta$. Since $\lambda = 1+\Delta$, the same snapping argument goes through. Therefore the conclusions of Theorem 3 and Corollary 4 remain valid for the discrete metric as well. The $p$-adic case remains the main one because it realises the same indicator behaviour inside the affine $p$-adic regression framework used throughout the paper.

**Boolean clause rewards.** Let

$$\Phi = \bigwedge_{r=1}^{m} C_r$$

be a CNF formula on variables $z_1,\dots,z_n$, with every clause of width at most $k$, and encode truth values by

$$x_i = 0 \iff z_i \text{ is true}, \qquad x_i = 1 \iff z_i \text{ is false}.$$

For a literal $\lambda$ on variable $z_i$, define its failure indicator by

$$\delta(\lambda; x) = \begin{cases} x_i, & \lambda = z_i, \\ 1-x_i, & \lambda = \neg z_i. \end{cases}$$

For a clause $C = (\lambda_1 \vee \cdots \vee \lambda_s)$, where $s \le k$,

$$C(x) \text{ is false} \iff \sum_{j=1}^{s} \delta(\lambda_j; x) = s.$$

After moving constants to the right-hand side, this means that there are $u_C \in \mathbb{Z}^n$ and $t_C \in \mathbb{Z}$ such that

$$C(x) \text{ is true} \iff u_C^\top x \neq t_C.$$

Here $u_C$ records the $\pm 1$ literal coefficients, and $t_C = s - \nu_C$, where $\nu_C$ is the number of negated literals in the clause. The translation is mechanical: a positive literal contributes $+x_i$, a negated literal contributes $-x_i$, and the forbidden right-hand side is $s - \nu_C$. For example,

$$(z_1 \vee z_2 \vee \neg z_3) \rightsquigarrow x_1 + x_2 - x_3 \neq 2.$$

If $p > k$ and $x \in \{0,1\}^n$, then $u_C^\top x - t_C$ is 0 when $C$ is false and belongs to $\{-1,\dots,-s\}$ when $C$ is true, so

$$|u_C^\top x - t_C|_p = \mathbf{1}_{C(x) \text{ is true}}.$$

Thus a negative observation $(-1, u_C, t_C, 1)$ rewards satisfaction of the clause. If $\Delta$ is the maximum number of clauses containing any variable and $\alpha > \Delta$, then

$$L_\Phi(x) = \alpha \sum_{i=1}^{n} \big(|x_i|_p + |x_i - 1|_p\big) - \sum_{C\in\Phi} |u_C^\top x - t_C|_p$$

has the same snapping property as before: every global minimiser lies in $\{0,1\}^n$, and on Boolean assignments

$$L_\Phi(x) = \alpha n - \mathrm{sat}_\Phi(x).$$

So the global minimisers are exactly the assignments that maximise the number of satisfied clauses. Unlike the earlier ordinary summed 2-adic MAX-CUT reduction [2], this construction uses a signed objective; its 3-CNF specialisation assumes $p > 3$.

**General false labels.** Nothing above requires the false value to be 1 for every variable. Encode $z_i =$ false by any $b_i \equiv 1 \pmod p$, not necessarily distinct, keep the $\pm 1$ literal coefficients, take as the forbidden target the sum of the positive literals' labels, and replace the Boolean wells by $|x_i|_p + |x_i - b_i|_p$. On Boolean-labelled assignments the clause residual equals minus the sum of the false labels of the satisfied literals: zero when the clause is falsified, and otherwise a sum of at most $k < p$ labels congruent to 1, hence a unit. The snapping and domination arguments are unchanged because every integer norm is at most 1. The uniform label $b_i = 1$ is the degenerate member of this family, and the experiments in this paper use it; Section 2.3 describes the member $b_i = 1 + p\,2^i$, which leaves the loss landscape identical while making each residual value identify the satisfied literals of its clause.

**Corollary 5** (NP-hardness at a fixed prime)**.** *The threshold decision problem for signed affine p-adic residual objectives is NP-hard already for the fixed prime $p = 5$; consequently, so is global minimisation.*

*Proof.* Apply the construction above to a 3-CNF formula, with $p = 5$ and $\alpha = \Delta + 1$. Its minimum is at most the threshold $\alpha n - m$ if and only if all $m$ clauses can be satisfied. This is a polynomial-time reduction from 3-SAT to the threshold problem. □

**Signed versus positive complement encodings.** The signed construction is compact rather than uniquely expressive. For the uniform label $b_i = 1$, the same clause can be encoded positively through its failure sum

$$s_C(x) := \sum_{j=1}^{s} \delta(\lambda_j; x)$$

and the allowed set $S_C = \{0, \ldots, s-1\}$. For $S \subseteq \{0, \ldots, s\}$, set

$$W_S(t) = \sum_{a \in S} |t - a|_p.$$

When $p > s$ and $t \in \{0, \ldots, s\}$, the positive term $W_{S_C}(t)$ equals $s - 1$ on the satisfying values $0, \ldots, s-1$ and $s$ on the forbidden value $s$. It therefore encodes exactly the same clause penalty using $s$ positive observations. By contrast, the signed encoding uses one negative observation for the single forbidden affine equality. Filling the missing positive well at $s$ makes the positive sum constant, which illustrates the complement relationship. The contribution of the signed form is the direct one-row representation of a small forbidden set, together with the coordinatewise domination theorem that keeps its negative reward bounded and snaps variables to their domains.

# 4 Minimum conflicts

## 4.1 Unsatisfiable instances

The characterisation holds even when the instance has no proper list-colouring: there the global minimisers are the domain-respecting assignments of least conflict count. For example, take the

triangle $K_3$ with all three lists equal to $\{1, 2\}$. No proper 2-colouring of $K_3$ exists, so every domain-respecting assignment leaves at least one edge monochromatic. Here $\Delta = 2$, so $\lambda = 3$, and by Theorem 3 the global minimisers are exactly the assignments with a single conflict, at loss $\lambda \sum_i (|D_i| - 1) - |E| + 1 = 3 \cdot 3 - 3 + 1 = 7$. This Max-CSP regime is distinct from the satisfiable case.

If a best-effort solution should treat some constraints as more important than others, assign them correspondingly higher or lower losses. Replacing the unit reward on an edge $e$ by a weight $w_e > 0$, and raising each pinning weight $\lambda_i$ above the total weight incident on coordinate $i$, gives the weighted minimum-conflict objective by the same domination argument.

# 5 Sudoku as a special case

Sudoku has standard exact-cover, constraint-programming, and SAT formulations [9, 10, 11], and puzzle variants have their own complexity theory [12]. The construction below is not meant to compete with those solvers. Sudoku is used because its variables, domains, and all-different constraints are small enough to write out explicitly. We then perform experiments with $p$-adic machine-learning training methods on a small sample of randomly carved Sudoku instances.

## 5.1 Sudoku

A (standard) Sudoku puzzle consists of a partially filled $9 \times 9$ grid. The task is to assign a digit in $\{1, \ldots, 9\}$ to each empty cell such that:

1. each row contains all digits $1, \ldots, 9$ exactly once;
2. each column contains all digits $1, \ldots, 9$ exactly once; and
3. each $3 \times 3$ box contains all digits $1, \ldots, 9$ exactly once.

Equivalently, in each unit (row, column, box) all entries are *pairwise distinct* and each entry lies in $\{1, \ldots, 9\}$.

Standard Sudoku fits Corollary 4 directly. The vertices are the 81 cells, two vertices are adjacent when the corresponding cells share a row, column, or $3 \times 3$ box, unclued cells have domain $\{1, \ldots, 9\}$, and clue cells have singleton domains. A proper list-colouring of this graph is exactly a valid Sudoku completion.

The Sudoku facts below are direct corollaries of the unary-well and edge-indicator lemmas. We state them before writing the concrete objectives.

## 5.2 Positive pinning: snapping coefficients to digits

Let $D = \{1, 2, \ldots, 9\} \subset \mathbb{Z}_p$, and write the Sudoku variables as a vector

$$x \in \mathbb{Z}_p^{81},$$

with one coefficient per cell.

**Definition 4** (Sudoku digit-snapping penalty)**.** *For $p > 9$, define the digit-snapping penalty*

$$U_D(t) := \sum_{k=1}^{9} |t - k|_p.$$

By Lemma 2 applied to the common domain $D$, one has $U_D(t) \geq 8$ for every $t \in \mathbb{Z}_p$, with equality if and only if $t \in D$.

The unary term is the nine-valued specialisation of the arbitrary-residue positive wells in Mihara's general forcing theorem [16]: the observations $t = k$ for all $k \in D$ create a multi-well objective with minima exactly at $D$. The additional signed terms below reward pairwise inequality and produce the Sudoku/list-colouring objective.

### 5.3 Negative signed residual terms: rewarding inequality via p-adic norms

The remaining Sudoku constraints are that entries in each row/column/box are pairwise unequal. Pairwise inequality can be encoded using differences $x_i - x_j$.

**Corollary 6** (Sudoku differences as inequality indicators)**.** *Let $p > 9$ and $a, b \in D$. Then*

$$|a-b|_p = \begin{cases} 0 & a = b, \\ 1 & a \neq b. \end{cases}$$

*Proof.* This is Lemma 1 with $q = 9$. □

### 5.4 Minimal and expanded residual objectives

Let $\mathcal{P}$ be the set of cell-pairs $(i, j)$ that share a unit (same row, same column, or same box). Let $\mathcal{C}$ be the set of given clues; for $i \in \mathcal{C}$ let $g_i \in D$ be the fixed digit.

Specialising Definitions 2 and 3, together with Theorem 3, to the Sudoku graph gives the minimal synthetic dataset

$$L_{\min}(x) = \alpha \sum_{i \notin \mathcal{C}} \sum_{k=1}^{9} |x_i - k|_p \; + \; \alpha \sum_{i \in \mathcal{C}} |x_i - g_i|_p \; - \sum_{(i,j) \in \mathcal{P}} |x_i - x_j|_p. \tag{1}$$

A separate coefficient on the pairwise term is unnecessary here: any positive edge weight can be scaled out, so we normalise that coefficient to 1. Thus $\alpha$ itself plays the role of the general pinning weight $\lambda$, and any choice $\alpha > 20$ satisfies the domination condition from Theorem 3. The minimal dataset therefore contains $9(81 - |\mathcal{C}|) + |\mathcal{C}| = 729 - 8|\mathcal{C}|$ positive observations and 810 negative observations. This is the formulation used in the correctness statements below.

For exposition, we also use the redundant expanded objective

$$L(x) = \alpha \sum_{i=1}^{81} \sum_{k=1}^{9} |x_i - k|_p \; + \; \alpha_{\text{clue}} \sum_{i \in \mathcal{C}} |x_i - g_i|_p \; - \sum_{(i,j) \in \mathcal{P}} |x_i - x_j|_p. \tag{2}$$

Equation (2) keeps the nine digit-well residuals visible even on clue cells and then adds the clue pins separately. It therefore contains $729 + |\mathcal{C}|$ positive observations and the same 810 negative observations. The first two sums are positive pinning terms, and the final sum is a negative pairwise reward term. Equivalently, counting peer pairs per unit gives $27\binom{9}{2} = 972$ negative terms, but 162 of these pairs lie simultaneously in a row/column *and* a box, leaving 810 distinct pairs overall.

This expanded form needs an explicit clue-weight assumption. On a clue cell, the first sum in (2) has the same value $8\alpha$ for every digit $x_i \in D$, so clue fidelity is enforced only by $\alpha_{\text{clue}}|x_i - g_i|_p$. The same domination argument applies to (2) when $\alpha_{\text{clue}} > 20$; the simplest choice is $\alpha_{\text{clue}} = \alpha = 21$. Under that extra condition, (2) is a redundant expanded form of (1).

Once $x$ is digit-valued, the pairwise part is simply

$$-\sum_{(i,j) \in \mathcal{P}} |x_i - x_j|_p = -|\mathcal{P}| + \sum_{(i,j) \in \mathcal{P}} \mathbf{1}_{x_i = x_j}.$$

For the minimal objective on domain-respecting states, and for the expanded objective on digit-valued, clue-consistent states, the remaining terms are constant. In those regimes the loss differs from the *deduplicated* peer-conflict count only by an additive constant. The heuristics below do not optimise that deduplicated peer objective directly: on the row-permutation state space they use a duplicated unit-scope column/box surrogate that counts conflicts separately inside each column and each box. That surrogate is described in Section 7 and Appendix B.

**Corollary 7** (Sudoku completion as a special case)**.** *Let $G_{\mathrm{Sud}}$ be the graph on the* 81 *cells of a Sudoku puzzle, with an edge between two cells exactly when they share a row, column, or* $3 \times 3$ *box. For a puzzle with clue set $\mathcal{C}$ and clue digits $g_i$, define*

$$D_i := \begin{cases} \{g_i\}, & i \in \mathcal{C}, \\ \{1, \dots, 9\}, & i \notin \mathcal{C}. \end{cases}$$

*Assume $\alpha > 20$. Then every global minimiser of the minimal objective* (1) *is a clue-respecting digit assignment that minimises the peer-conflict count. In particular, if the Sudoku instance is satisfiable, the global minimisers are exactly the valid completions of the puzzle.*

*Proof.* Each row, column, and box of Sudoku forms a clique of size 9 in $G_{\mathrm{Sud}}$. Thus a proper list-colouring assigns pairwise distinct digits in each unit. Since the available values are exactly $\{1, \dots, 9\}$, pairwise distinctness in a unit is equivalent to containing every digit exactly once. The singleton domains enforce the clues. Apply Theorem 3. □

In the deduplicated peer graph, every cell has degree 20, so Theorem 3 requires $\alpha > 20$ for the minimal objective (1); $\alpha = 21$ is enough. For the expanded objective (2), the same claim requires $\alpha_{\mathrm{clue}} > 20$, because the all-digit wells are constant on clue cells. If one instead counts row, column, and box overlaps separately, each cell has $8 + 8 + 8 = 24$ incident negative terms, and the safe unary bound becomes $> 24$.

## 6 Representational parsimony

A standard SAT or exact-cover lift introduces Boolean variables

$$y_{r,c,d} \in \{0, 1\},$$

with the intended meaning that cell $(r, c)$ contains digit $d$. This produces $9 \times 9 \times 9 = 729$ indicators before any clues are applied. The p-adic construction instead uses an 81-cell digit space. This variable-level comparison is the main parsimony claim: one digit-valued parameter per semantic object rather than nine Boolean indicators per cell.

The minimal objective (1) is already a single scalar loss on that state space. The theorem-side expanded objective (2) contains 729 digit-snapping residuals, $|\mathcal{C}|$ clue residuals, and 810 pairwise residuals. A structured constraint-programming model with 81 cell variables and 27 *alldifferent* constraints is already semantically close to the puzzle, so the comparison here is not based on raw constraint count alone. The signed $p$-adic construction packages the all-different structure into one residual objective on the 81 cell variables.

Appendix D records an alternative encoding that maps digits to powers of two so that each row, column, or box contributes a single sum residual rather than pairwise terms. In local-search experiments on 50 benchmark puzzles, that encoding produced no successful solves: the search repeatedly stopped in local minima.

## 7 Computations with the induced $p$-adic loss

We implemented two finite-state local-search solvers: a greedy row-swap search and a discrete Zubarev walk. We also implemented a Mihara-inspired last-digit (modulo-$p$) equality fit as a diagnostic comparison. Appendix B defines the solver loss, records the two algorithms and a locality lemma for row swaps, and gives traces on a standard puzzle.

The greedy solver simply counts integer conflicts: it does not evaluate a $p$-adic valuation numerically. On the digit-restricted state space this count is an exact integer representative of the induced zero-or-one residual loss, with column–box overlaps counted twice. The Zubarev

solver uses the same induced loss through the Boltzmann transition rule $\exp(-\beta\,\Delta L)$; its finite implementation can likewise compute $\Delta L$ from integer conflicts because all relevant norms have already collapsed to indicators. The Mihara-inspired comparison instead performs modulo-19 last-digit equality fitting on a positive-only expansion: every negative peer row is replaced by the sixteen equalities for nonzero digit differences, and unary-row weights are preserved in the consensus score.

On a small sample of 18 randomly carved puzzles, the greedy and Zubarev solvers both completed every puzzle, giving 36 successful method–puzzle runs. The row-swap search used fewer steps on 14 of the 18 paired puzzles. Its median step counts were 537.5, 2784, and 7040 at 36, 30, and 26 clues, versus 4533.5, 8751.5, and 9911 for the Zubarev walk. Appendix C gives the experimental setup, full table, and a representative loss curve. These smoke tests show only that both finite-state procedures can reach completions in this sample; they are not a solver benchmark. The Mihara-inspired solver is not included among the 36 solver runs, since it did not successfully solve any of the sample puzzles.

The solver is available at `https://padic-logic.symmachus.org/` (with source code at `https://github.com/solresol/padic-logic`. The code that ran these experiments is at `https://github.com/solresol/sudoku-padic-regression`.


Figure 3: The Sudoku route after a row-swap run reaches a valid completion. The native 81-cell grid, the 1,299-row signed objective summary, the theoretical-floor equality, and the zero-conflict search result remain visible together.

# 8 Related work and scope

## 8.1 Earlier regression and constraint models

From the constraint-programming side, the finite-domain objective is a valued-CSP or Max-CSP penalty function [3]. Replacing a global *alldifferent* constraint by a clique of pairwise inequalities preserves its solutions but not the stronger propagation of a matching-based global constraint [4]. On the finite domain, the list-colouring objective is the zero-temperature antiferromagnetic Potts energy with local list constraints [5]; related discrete problems are often mapped to Ising or QUBO objectives [6]. The local-search experiments are closest algorithmically to min-conflicts repair search [7] and to Lewis's metaheuristic Sudoku study [8], whose simulated annealing keeps each $3 \times 3$ box as a permutation of $1, \ldots, 9$ and swaps two non-clue cells within a box; the heuristics in Appendix B make the analogous choice with rows as the permutation unit. Sudoku itself is routinely formulated using exact cover, SAT, or *alldifferent* constraints [9, 10, 11]. Martins gives a broader recent account of learning primitives over $p$-adic spaces [14]; the present construction specialises instead to signed affine objectives that compile finite-domain constraints.

# 9 Conclusion

Signed $p$-adic residuals have a direct finite-domain role. Positive synthetic terms pin variables to a finite set; after that, negative terms are bounded rewards for inequality or clause satisfaction.

Sudoku is a finite-domain case study in which the variables, domains, and all-different constraints are explicit. All-different systems admit polynomial-time encodings whose global minimisers are exactly the domain-respecting minimum-conflict assignments, and in the satisfiable case exactly the satisfying assignments.

The same signed-residual construction is not limited to pairwise inequality. After Boolean pinning, every clause of width less than $p$ becomes one forbidden affine equality, so a single signed term rewards its satisfaction. A positive-only affine encoding can represent the same finite-domain penalty by listing all allowed residual values; the signed formulation's advantage is the compact one-row representation of the forbidden complement within that affine class. Section 2.3 shows how variables can receive distinct false labels whose residual values identify the satisfied literals without changing the loss landscape, and Appendix G records a polynomial-valued variant. No optimisation or verification advantage is claimed for these labellings.

The companion site, `https://padic-logic.symmachus.org`, lets readers trace a Boolean CSP from clauses to forbidden affine equalities and a signed residual dataframe, or compile an editable Sudoku into its 1,299-row objective and watch search reach the theoretical floor.

# A Mihara's digitwise regression and positive complement expansion

Mihara's forcing theorem [16] considers a product $C = \prod_i B_i$ of residue sets, a nonnegative base objective satisfying a concentration condition near $C$, and a positive unary forcing term whose weight $M$ exceeds the base objective's range on $C$. The forced objective has the same optima on $C$; specialising the construction to affine residual data proves ordinary positive $p$-adic linear regression NP-hard for every fixed prime.

| | Mihara's forcing theorem | Signed affine compiler here |
|---|---|---|
| Base terms | Nonnegative $p$-adic optimisation concentrated near a product of residue sets | Signed affine residuals that are zero or units on the target finite domain |
| Pinning condition | One forcing scale $M$ above the base objective's range on the finite set | Coordinatewise $\lambda_i$ above the absolute incident interaction weight |
| Off-domain control | Concentration plus positive unary forcing | Ultrametric Lipschitz bound for positive and negative interaction rows |
| Finite-domain role | Preserve the base objective's optima on $C$ | Realise equality, inequality, and forbidden-hyperplane indicators exactly |
| Main consequence | Positive linear regression NP-hard for each fixed prime | Compact complement encoding and exact minimum-conflict characterisation |

Table 5: Comparison with Mihara's generalisation of Baker forcing [16]. The two results share finite-domain pinning but use different hypotheses to control the remaining objective.

Mihara's linear-regression algorithm is described in [15]. For his digitwise regression algorithm, the data are samples

$$(\vec{x}_i, y_i) \in \mathbb{Z}_p^D \times \mathbb{Z}_p$$

from a single hidden affine graph

$$y = \langle c, \vec{x} \rangle,$$

with digitwise noise. Writing

$$I_e = \{i : y_i - \langle c, \vec{x}_i \rangle \in p^e \mathbb{Z}_p\},$$

the method estimates $c \bmod p^E$ recursively: first solve a modulo-$p$ linear-regression problem to estimate $c \bmod p$, subtract that digit, divide residuals by $p$, restrict to the samples in $I_1$, and repeat. The probabilistic argument relies on a large, sufficiently random sample from the hidden affine graph, a small digitwise corruption rate such as

$$\frac{|I_e \setminus I_{e+1}|}{|I_e|} \leq r \ll \frac{1}{2},$$

and the condition that the noise-free samples have the right affine hull modulo $p$ at each digit step.

Each finite-field elimination, consistency check, and dataset scan between random draws is polynomial-time. The complete published procedure is not worst-case polynomial-time: Algorithm 6 has an outer `while True` restart loop, while its sampling subroutines have no polynomial bound on the number of random draws. Mihara explicitly notes that the process need not terminate and reports one $D = 100$, $r = 0.1$ experiment in which it had not terminated after 1,700 initialisations. The probability analysis gives useful expected behaviour when the random-sampling, affine-hull, and noise assumptions hold; it does not provide a polynomial termination guarantee for arbitrary input data.

Those assumptions do not directly match the signed Sudoku construction. In this paper the unknowns $x_i$ are the cell values themselves, not coefficients of an observed response law. The residual rows are exact synthetic constraints chosen by the modeller: unary wells pin cells to allowed digits, while negative residuals $-|x_i - x_j|_p$ reward inequality across peer edges. After digit snapping, the problem is a finite-domain conflict-minimisation problem. There is no naturally sampled response vector $Y$, no hidden coefficient vector $c$, and no large noise-free affine locus whose graph is assumed to have generated the observations.

For a finite-domain negative row, however, there is an exact positive-only reformulation. Let

$$S = \{u^\top x : x \in C\}$$

be the attainable affine values on the forced finite domain $C$, and let $t \in S$ be the forbidden target. When distinct elements of $S$ remain distinct modulo $p$,

$$\sum_{s \in S \setminus \{t\}} |u^\top x - s|_p = (|S| - 1) - |u^\top x - t|_p, \qquad x \in C.$$

Indeed, at $u^\top x = t$ all $|S| - 1$ positive rows contribute one, whereas at an allowed value exactly one complementary row vanishes. Thus the positive family has exactly the same minimisers as the original negative row, differing only by the constant $|S| - 1$.

For a CNF clause, $S_C = \{u_C^\top x : x \in \{0,1\}^n\}$ contains at most one more value than the clause width. Replacing the forbidden row $u_C^\top x = t_C$ by the positive rows $u_C^\top x = s$ for $s \in S_C \setminus \{t_C\}$ is therefore a polynomial-size expansion. Together with the positive weighted Boolean wells, it gives Mihara's modulo-$p$ stage a positive-only weighted-consensus dataset: a satisfying clause contributes one unit of inlier weight, while a failed clause contributes none. The original signed row is retained for reporting $v_p(u_C^\top x - t_C)$, its norm, and its signed contribution to the displayed loss.

For a Sudoku peer row, $S = \{-8, -7, \ldots, 7, 8\}$ and the forbidden target is 0. The positive complement therefore contains the sixteen equalities $x_i - x_j = s$ with $s \neq 0$. Choosing $p = 19$ keeps those integer differences distinct modulo $p$. On digit states an unequal peer satisfies exactly one complementary equality and an equal peer satisfies none. The browser preserves the pinning weight $\alpha = 21$, scores candidate vectors by total positive weight minus weighted inlier

count, and retains the best score across random full-rank bases. This weighting, complement expansion, grouped basis sampler, and best-consensus retention are adaptations made for the browser comparison; they are not claimed as steps of Mihara's published pseudocode. The browser implements only a last-digit modulo-$p$ fit, not the recursive recovery of coefficients modulo $p^E$. It separately evaluates the original signed rows, including the counts with $v_{19}(x_i - x_j) = 0$ and with zero residual.

The companion browser uses the positive complement expansion in both its Boolean-CSP and Sudoku Mihara modes. For Sudoku it plots two histories: the non-increasing positive-consensus loss used to select fits and the original signed objective used only as an audit. A better consensus fit can still be off-domain or violate clues, and the signed audit need not decrease. The interface therefore continues fresh starts until a decoded vector satisfies the original constraints or the user stops it. It treats the procedure as an unbounded-restart heuristic rather than a polynomial-time solver. A method guaranteed to solve every compiled instance in polynomial time would decide the NP-hard threshold problem of Corollary 5; Mihara's algorithm makes no such guarantee.

# B Heuristic details

## B.1 Row-swap local search

To keep local updates cheap, the greedy implementation does not evaluate (2) or any $p$-adic valuation numerically. Instead, on the digit-valued, clue-consistent row-permutation state space it minimises the duplicated unit-scope column/box conflict count

$$H_{\mathrm{cb}}(x) := \sum_{c=1}^{9} \mathrm{conf}(C_c; x) \; + \; \sum_{b=1}^{9} \mathrm{conf}(B_b; x),$$

where $C_c$ and $B_b$ are the cells in column $c$ and box $b$, and

$$\mathrm{conf}(U; x) := \sum_{d=1}^{9} \binom{\#\{u \in U : x_u = d\}}{2}.$$

Because each row is kept as a permutation of $\{1, \ldots, 9\}$, the corresponding row contribution is identically zero and is omitted. This duplicated unit-scope loss is close to, but not identical with, the deduplicated peer-conflict objective coming from $\mathcal{P}$: a conflicting pair that lies simultaneously in a column and a box is counted twice. For the seed-0 initialisation shown later in Figure 5, the deduplicated peer-conflict count is 47 whereas $H_{\mathrm{cb}} = 56$, because nine conflicting pairs lie in both a column and a box.

The heuristics work directly on the digit-valued, clue-consistent row-permutation state space and minimise $H_{\mathrm{cb}}$. Concretely, we initialise each row as a random permutation of $1, \ldots, 9$ consistent with its clues, and then apply row-wise swaps that reduce $H_{\mathrm{cb}}$. This is a block-local search on the integer state space. The state space and neighbourhood follow the standard metaheuristic Sudoku design of Lewis [8], with rows rather than boxes as the permutation unit; the search procedure itself is not a contribution of this paper.

**Lemma 4** (Locality of row swaps)**.** *Let $x$ be a digit-valued, clue-consistent Sudoku state in which each row is a permutation of $1, \ldots, 9$. Let $\xi$ swap two non-clue entries in a single row $r$, at columns $c_1 \neq c_2$. Then the change*

$$\Delta H_{\mathrm{cb}}(\xi) := H_{\mathrm{cb}}(x \oplus \xi) - H_{\mathrm{cb}}(x)$$

*is determined entirely by the conflict-pair contributions from columns $c_1$ and $c_2$, together with the one or two boxes containing $(r, c_1)$ and $(r, c_2)$. No other term of the digit-restricted objective changes.*

**Algorithm 1** Row-swap local search

**Require:** Puzzle with clues $\mathcal{C}$, max steps $T$, restarts $R$

**for** $r = 1$ to $R$ **do**
  Initialise each row as a permutation of $1, \ldots, 9$ consistent with its clues
  **for** $t = 1$ to $T$ **do**
    **if** no column/box conflicts **then**
      **return** solution
    **end if**
    Choose a row involved in a conflict
    Find the best swap of two non-clue cells in that row (largest decrease in $H_{\text{cb}}$)
    Apply the swap (or a random swap if no improving swap exists)
  **end for**
**end for**
**return** best assignment found

*Proof.* On the row-permutation state space, the heuristic objective is exactly $H_{\text{cb}}$, so only column and box contributions matter. A within-row swap preserves the row permutation, so the omitted row contribution remains zero. Every cell outside row $r$ is unchanged, and within row $r$ only the two swapped positions change value. Hence only the two corresponding columns can change their conflict counts. Likewise, only the boxes containing those two cells can change. All other columns and boxes see the same multiset of digits before and after the swap, so their contributions are unchanged. □

### B.2 Trace on a standard puzzle

To make the row-swap heuristic concrete, Figure 4 shows the standard example puzzle used in many Sudoku tutorials. With seed 0 (restart 0), the row-wise random initialisation used by the solver produces the filled grid in Figure 5; this state has 56 column/box conflict pairs. Table 6 shows the first few swaps from that initial state.

Step 2 illustrates the locality claim. After Step 1, row 6 contains

$$(7, 3, 1, 8, 2, 5, 4, 9, 6),$$

and the chosen move swaps the entries in columns 3 and 6, exchanging 1 and 5. By Lemma 4, only column 3, column 6, the box on rows 4–6 and columns 1–3, and the box on rows 4–6 and columns 4–6 need to be inspected. Before the swap these four units contribute

$$4 + 6 + 1 + 2 = 13$$

conflict pairs; afterwards they contribute

$$3 + 3 + 2 + 1 = 9.$$

Thus $\Delta H_{\text{cb}} = -4$, exactly as reported in Table 6. For this instance the solver reaches a full solution after 227 swaps on the same restart.

### B.3 A discrete Zubarev walk

The row-swap heuristic above is tailored to the strong-snapping Sudoku setting. It is also natural to test a more generic training dynamic from the p-adic regression literature. In [13], Zubarev proposes a random-walk-based optimisation scheme for loss functions built from p-adic norms, precisely because such losses are locally constant almost everywhere and hence poorly served by gradient methods.

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 5 | 3 | | | 7 | | | | |
| 6 | | | 1 | 9 | 5 | | | |
| | 9 | 8 | | | | | 6 | |
| 8 | | | | 6 | | | | 3 |
| 4 | | | 8 | | 3 | | | 1 |
| 7 | | | | 2 | | | | 6 |
| | 6 | | | | | 2 | 8 | |
| | | | 4 | 1 | 9 | | | 5 |
| | | | | 8 | | | 7 | 9 |

Figure 4: Standard example puzzle used for the heuristic traces.

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 5 | 3 | 8 | 4 | 7 | 2 | 1 | 9 | 6 |
| 6 | 2 | 8 | 1 | 9 | 5 | 3 | 4 | 7 |
| 3 | 9 | 8 | 1 | 4 | 7 | 2 | 6 | 5 |
| 8 | 2 | 9 | 5 | 6 | 4 | 7 | 1 | 3 |
| 4 | 5 | 7 | 8 | 9 | 3 | 2 | 6 | 1 |
| 7 | 3 | 1 | 8 | 2 | 5 | 4 | 9 | 6 |
| 3 | 6 | 1 | 7 | 9 | 5 | 2 | 8 | 4 |
| 2 | 6 | 3 | 4 | 1 | 9 | 7 | 8 | 5 |
| 3 | 2 | 4 | 6 | 8 | 5 | 1 | 7 | 9 |

Figure 5: Seed-0 row-wise random initialisation (restart 0), before any swaps.

Concretely, the proposed training dynamics is a discrete-time random process

$$w_{t+1} = w_t + \xi_t(w_t, \beta_t), \tag{3}$$

where $\beta_t > 0$ is a "continuity" (inverse temperature) parameter and $\xi_t$ is sampled from a distribution biased toward loss-decreasing moves. In Zubarev's formulation the conditional law has density (with respect to the $p$-adic Haar measure)

$$P(\xi \mid w, \beta) \;\propto\; \exp\big(-\beta\,(L(w+\xi) - L(w))\big). \tag{4}$$

Under mild conditions (in particular, that the set of improving moves has nonzero measure), Zubarev shows there is a regime of $\beta$ values for which the expected loss decreases, i.e. $L(w_t)$ is a strict supermartingale [13].

In the digit-restricted implementation used here, the state is already digit-valued, clue-consistent, and row-permuted. We therefore treat the Zubarev walk as a *finite-move* stochastic optimiser:

- The parameter vector $w$ is the current Sudoku filling $x \in D^{81}$.
- The loss is the duplicated unit-scope column/box count $H_{\mathrm{cb}}(x)$.

| Step | Row | Swap (columns) | Swap (digits) | Type | Conflicts (before) | (after) |
|---|---|---|---|---|---|---|
| 1 | 5 | $7 \leftrightarrow 8$ | $2 \leftrightarrow 6$ | best | 56 | 53 |
| 2 | 6 | $3 \leftrightarrow 6$ | $1 \leftrightarrow 5$ | best | 53 | 49 |
| 3 | 2 | $2 \leftrightarrow 3$ | $2 \leftrightarrow 8$ | best | 49 | 45 |
| 4 | 4 | $7 \leftrightarrow 6$ | $7 \leftrightarrow 4$ | random | 45 | 47 |
| 5 | 7 | $1 \leftrightarrow 5$ | $3 \leftrightarrow 9$ | best | 47 | 40 |
| 6 | 4 | $6 \leftrightarrow 7$ | $7 \leftrightarrow 4$ | best | 40 | 38 |
| 7 | 2 | $2 \leftrightarrow 9$ | $8 \leftrightarrow 7$ | best | 38 | 36 |
| 8 | 1 | $3 \leftrightarrow 4$ | $8 \leftrightarrow 4$ | best | 36 | 35 |

Table 6: First eight swap steps for the standard example puzzle (seed 0). The table records both the swapped column indices and the digits occupying those columns before each move. Rows and columns are 1-indexed.

**Algorithm 2** Zubarev walk over row swaps (heuristic)

**Require:** Puzzle with clues $\mathcal{C}$, max steps $T$, restarts $R$, schedule $\{\beta_t\}_{t=1}^{T}$
**for** $r = 1$ to $R$ **do**
  Initialise each row as a permutation of $1, \ldots, 9$ consistent with its clues
  **for** $t = 1$ to $T$ **do**
    **if** no column/box conflicts **then**
      **return** solution
    **end if**
    Choose a row involved in a conflict
    Enumerate row swaps $\Xi_r(x)$ and compute $\Delta H_{\mathrm{cb}}(\xi)$ for each $\xi \in \Xi_r(x)$
    Sample $\xi \in \Xi_r(x)$ with probability $\propto \exp(-\beta_t \, \Delta H_{\mathrm{cb}}(\xi))$
    Apply swap $\xi$
  **end for**
**end for**
**return** best assignment found

- A move $\xi$ is a swap of two non-clue cells within a single row (preserving each row as a permutation of $1, \ldots, 9$).

Replacing Haar measure and a continuous $p$-adic state space by counting measure on row swaps changes the hypotheses of Zubarev's result. No supermartingale or convergence guarantee is claimed for the finite heuristic below; the Boltzmann weighting is used only as a motivated move-selection rule.

For a chosen row $r$ and current state $x$, let $\Xi_r(x)$ be the set of all such swaps, and let $\Delta H_{\mathrm{cb}}(\xi) := H_{\mathrm{cb}}(x \oplus \xi) - H_{\mathrm{cb}}(x)$ be the change in loss after applying swap $\xi$ (where $x \oplus \xi$ denotes "apply the swap"). Replacing the Haar measure in (4) with the counting measure on the finite set $\Xi_r(x)$ gives the discrete sampling rule

$$\Pr(\xi \mid x, \beta_t) = \frac{\exp\big(-\beta_t \, \Delta H_{\mathrm{cb}}(\xi)\big)}{\sum_{\xi' \in \Xi_r(x)} \exp\big(-\beta_t \, \Delta H_{\mathrm{cb}}(\xi')\big)}. \tag{5}$$

When $\beta_t \to 0$ this is uniform random swapping; when $\beta_t \to \infty$ it concentrates on the best (most loss-decreasing) swap, recovering a soft version of the greedy stepwise algorithm. In practice we use a simple annealing schedule: $\beta_t$ increases linearly from $\beta_0$ to $\beta_1$ over the allotted steps, gradually shifting from exploration to exploitation.

| Step | Row | Swap (columns) | Conflicts (before) | (after) |
|---|---|---|---|---|
| 1 | 5 | $2 \leftrightarrow 8$ | 56 | 56 |
| 2 | 6 | $7 \leftrightarrow 8$ | 56 | 56 |
| 3 | 2 | $3 \leftrightarrow 7$ | 56 | 55 |
| 4 | 8 | $2 \leftrightarrow 8$ | 55 | 51 |
| 5 | 4 | $2 \leftrightarrow 3$ | 51 | 50 |
| 6 | 4 | $6 \leftrightarrow 7$ | 50 | 51 |
| 7 | 2 | $7 \leftrightarrow 8$ | 51 | 52 |
| 8 | 8 | $3 \leftrightarrow 7$ | 52 | 49 |

Table 7: First eight Zubarev-walk swap steps for the standard example puzzle (seed 0). Rows and columns are 1-indexed.

### B.4 Zubarev trace on the standard puzzle

Using the same standard example puzzle, the Zubarev walk (seed 0, restart 0, $\beta_0 = 0.5$, $\beta_1 = 6.0$ linear schedule) starts from the same row-consistent initialisation with 56 column/box conflict pairs. Unlike the greedy stepwise method, the walk sometimes accepts loss-increasing or loss-neutral swaps early on; Table 7 shows the first few moves. For this instance the Zubarev walk reaches a full solution after 10,029 swaps on the same restart.

## C Illustrative experiments

We implemented both solvers above in JavaScript for the website and in pure Python; the experiment code is available at `https://github.com/solresol/sudoku-padic-regression`. The prime was set to $p = 11$, but in the digit-restricted regime the relevant residual norms are exactly zero-or-one indicators. The greedy solver evaluates the resulting integer conflict count $H_{\mathrm{cb}}$ directly. The Zubarev solver uses changes in the same induced loss in its Boltzmann probabilities; those changes can also be computed from $H_{\mathrm{cb}}$ without numerically calling a valuation routine. Thus the $p$-adic construction determines the finite-state loss, while the implementation uses its exact integer representation.

### C.1 Experimental setup

To obtain a quick, reproducible testbed, we generated random puzzles by:

1. sampling a random solved grid; then
2. carving it down to a specified clue count by removing entries uniformly at random.

This procedure does *not* enforce uniqueness of the resulting puzzle. Since Theorem 3 characterises the entire set of global minimisers as the set of valid completions, a puzzle with several completions has several global minimisers, and the heuristics report whichever one their trajectory reaches first. Each carved puzzle was then solved with up to $T = 60{,}000$ steps and $R = 15$ random restarts. All results reported below use a fixed RNG seed (123) for reproducibility. The two methods receive the same 18 puzzle seeds and the same corresponding solve seeds, so their results are paired. There is only one solve seed per method–puzzle pair; the sample is not difficulty-normalised and supports no inferential performance claim.

The table can be regenerated from the repository root with the following commands:

```
python3 code/run_experiments.py --outdir outputs/paper_stepwise \
  --seed 123 --n 6 --clues 36,30,26 --max-steps 60000 \
  --restarts 15 --method stepwise
```

| Method | Clues | Puzzles | Solved | Median steps | Median time (s) |
|---|---|---|---|---|---|
| Stepwise | 36 | 6 | 6 | 537.5 | 0.270 |
| Stepwise | 30 | 6 | 6 | 2784 | 1.315 |
| Stepwise | 26 | 6 | 6 | 7040 | 3.990 |
| Zubarev walk | 36 | 6 | 6 | 4533.5 | 2.091 |
| Zubarev walk | 30 | 6 | 6 | 8751.5 | 3.951 |
| Zubarev walk | 26 | 6 | 6 | 9911 | 4.786 |

Table 8: Descriptive smoke-test results on 18 paired, randomly carved puzzles (not uniqueness-checked), using $T = 60{,}000$ steps and $R = 15$ restarts. The Zubarev walk uses a linear schedule $\beta_0 = 0.5$ to $\beta_1 = 6.0$.

```
python3 code/run_experiments.py --outdir outputs/paper_zubarev \
  --seed 123 --n 6 --clues 36,30,26 --max-steps 60000 \
  --restarts 15 --method zubarev --beta0 0.5 --beta1 6.0 \
  --beta-schedule linear
```

### C.2 Results

Table 8 summarises 36 method–puzzle runs over 18 paired puzzles (six puzzles at each clue count, evaluated by both heuristics). Both methods succeeded on every puzzle. The row-swap method took fewer steps on 14/18 paired runs, including 5/6, 6/6, and 3/6 puzzles at 36, 30, and 26 clues respectively. Runtimes are machine-dependent and noisy, and the sample is too small for stronger algorithmic claims.

Figure 6 shows the conflict trajectory for the stepwise row-swap heuristic on the first 30-clue table instance, with puzzle seed 115642242 (solve seed 2738956839), using the row-wise random initialiser described above, $T = 60{,}000$, and $R = 15$. The plotted run is restart 0, and the vertical axis is the duplicated column/box conflict count $H_{\mathrm{cb}}$. The corresponding trace puzzle, solution, CSV row, and figure source are all included in `outputs/paper_stepwise/` in the submission archive.

## D Powers-of-two experiments

An alternative maps digits $d \in \{1, \ldots, 9\}$ to powers of two,

$$d \longmapsto 2^{d-1} \in A := \{1, 2, 4, \ldots, 256\}.$$

If every cell variable is restricted to $A$, then the condition that a row, column, or box contains each digit exactly once is equivalent to a single sum constraint

$$\sum_{(r,c)\in g} \beta_{r,c} = 1 + 2 + \cdots + 256 = 2^9 - 1 = 511,$$

for each unit $g$. This replaces the pairwise penalties by 27 residuals of the form

$$r_g(\beta) := 511 - \sum_{(r,c)\in g} \beta_{r,c}.$$

For a prime $p$, one can then study the objective

$$L_p(\beta) := \sum_g |r_g(\beta)|_p,$$

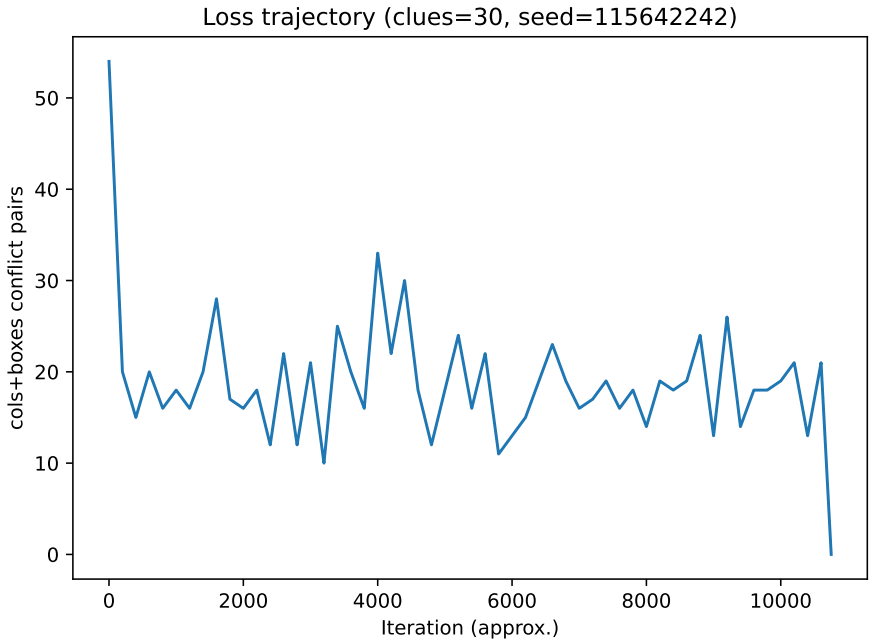


Figure 6: Conflict trajectory for the stepwise row-swap heuristic on the paired-table 30-clue instance with puzzle seed 115642242 (solve seed 2738956839), from the row-wise random initialisation on restart 0.

optionally together with a regulariser that encourages $\beta_{r,c} \in A$ and enforces the clues.

Archived local-search experiments with this encoding were run on 50 Euler Project Problem 96 puzzles. Table 9 summarises the outcomes. None of these runs found a valid completion.

| Experiment | Setting | Solved | Summary |
|---|---|---|---|
| E1 | Greedy cell-swap on 15 primes, 50 puzzles, 3 random initialisations per puzzle–prime pair (2250 runs) | 0/2250 | Mean lift 0.003; mean 12.5 steps |
| E2 greedy | Greedy cell-swap on the 5 strongest E1 primes, again with 50 puzzles and 3 initialisations (750 runs) | 0/750 | Mean lift 0.009; mean 19.3 steps |
| E2 SA | Simulated annealing on the same 750 runs | 0/750 | Mean lift 0.000; mean 5000 steps |

Table 9: Archived local-search results for the powers-of-two encoding. The E1 prime sweep used primes $2, 3, 5, 7, 11, 13, 17, 23, 47, 73, 97, 127, 257, 521, 2311$.

## E False-label and coefficient requirements

Section 2.3 used false labels $b_i \equiv 1 \pmod{p}$ with the standard $\pm 1$ literal coefficients. This appendix records the full set of conditions under which a signed clause row rewards satisfaction by exactly one unit, allowing both the label $b_i$ and the clause coefficients to be chosen freely, and shows that the two freedoms are the same freedom.

Fix a clause $C$ of width $s$. As before a variable's true value is $x_i = 0$ and its false value is $x_i = b_i$; now give literal $i$ of $C$ an integer coefficient $c_{C,i}$, and write $\epsilon_i = -1$ when the literal is positive and $\epsilon_i = +1$ when it is negated. Take the forbidden target $t_C = \sum_{\substack{i \in C \\ \text{positive}}} c_{C,i} b_i$. On a

Boolean-labelled assignment the affine residual is

$$u_C^\top x - t_C = \sum_{i \in \mathrm{Sat}(x)} w_{C,i}, \qquad w_{C,i} := \epsilon_i \, c_{C,i} \, b_i,$$

where $\mathrm{Sat}(x) \subseteq C$ is the set of satisfied literals: a satisfied positive literal has $x_i = 0$ and contributes $-c_{C,i}b_i$, a satisfied negated literal has $x_i = b_i$ and contributes $+c_{C,i}b_i$, and a falsified literal contributes 0. The unique falsifying assignment has $\mathrm{Sat}(x) = \varnothing$ and residual 0 for every choice of labels and coefficients.

Two conditions make the negative row $(-1, u_C, t_C, 1)$ reward satisfaction exactly.

(R1) *Each $b_i$ is a $p$-adic unit.* Then the well $|x_i|_p + |x_i - b_i|_p$ attains its minimum value 1 exactly on $x_i \in \{0, b_i\}$, and the pinning and domination arguments of Section 3 apply verbatim.

(R2) *Each clause is zero-sum-free modulo $p$:* no nonempty subset of $\{w_{C,i} \bmod p : i \in C\}$ sums to 0. Then every satisfying assignment yields a nonempty subset sum that is a unit, so $|u_C^\top x - t_C|_p = \mathbf{1}_{C(x)\ \mathrm{true}}$ and $L_\Phi(x) = \alpha n - \mathrm{sat}_\Phi(x)$ as in Section 3.

Applied to singletons, (R2) forces each $w_{C,i}$, hence each coefficient $c_{C,i}$, to be a unit, so $|c_{C,i}x_i|_p = |x_i|_p$ as the domination step requires. Because (R2) constrains only the residues $w_{C,i} = \epsilon_i c_{C,i} b_i \bmod p$, the label and the coefficient enter solely through their product: choosing $b_i$ and choosing $c_{C,i}$ are interchangeable up to that product.

A convenient sufficient form of (R2) is to choose representatives $r_i \in \{1, \ldots, p-1\}$ of $w_{C,i} \bmod p$ with $\sum_{i \in C} r_i < p$; every nonempty subset then sums to a positive integer below $p$. The uniform labelling $b_i = 1$ (all $r_i = 1$) is the case $s < p$, and $b_i = 2^i$ with per-clause label-sum below $p$ is another.

Failing (R2) causes obvious problems. Over $p = 5$ with labels $b_1 = 1$, $b_2 = 4$, the clause $(z_1 \vee z_2)$ has $b_1 + b_2 = 5 \equiv 0$: an assignment satisfying both literals has residual $-5$ and is rewarded $|-5|_5 = \frac{1}{5}$ instead of 1, so a doubly-satisfied clause counts as almost falsified and the loss stops tracking the satisfied-clause count.

**Correctness modulo $p$, information in the lift.** Conditions (R1) and (R2) depend only on residues modulo $p$, and on Boolean-labelled assignments the loss $L_\Phi$ depends only on the dataframe reduced modulo $p$, while the off-domain snapping argument uses only the fact that the relevant quantities are units. The integer dataframe is therefore best regarded as a choice of lift of a fixed matrix over $\mathbb{F}_p$: every lift of the same reduced dataframe defines the same objective on the finite domain, and the higher $p$-adic digits of the lift are free bandwidth. The uniform construction takes the trivial lift; the labels $b_i = 1 + p\,2^i$ of Section 2.3, the per-clause coefficients below, and the substitution of Appendix F are different lifts of the same reduced dataframe, chosen so that residual values carry useful information. Correctness lives in digit zero; information lives in the lift.

**Decoupling and per-clause provenance.** The label $b_i$ is global—shared by every clause on variable $i$ and by that variable's wells—whereas $c_{C,i}$ is local to one clause row. The coefficient freedom can therefore carry per-clause information without perturbing the wells or the search landscape. Fixing $b_i = 1$ and choosing $c_{C,i}$ so that $w_{C,i} \equiv 2^j \pmod p$, where $j$ indexes the literal within its clause, makes the residual of $C$ congruent modulo $p$ to the bitmask of $\mathrm{Sat}(x)$; the satisfied literals of that clause are read directly off the residual. This needs only $p > 2^s$, independently of the number of variables, in contrast to the global labels $b_i = 1 + p\,2^i$ of Section 2.3, whose size grows with the variable index.

**Random coefficients as residual fingerprints.** The coefficient freedom can be exercised by sampling instead of design. Draw each residue $w_{C,i}$ uniformly from the units of $\mathbb{F}_p$. Conditioning on all but one variable of a nonempty subset shows each subset sum vanishes with probability at most $1/(p-1)$, so (R2) fails with probability at most $(2^s-1)/(p-1)$ per clause; a draw is checked directly and resampled on failure. More useful is the pairwise guarantee: for distinct candidate sets $S \neq S'$ of satisfied literals, the residuals $\sum_{i\in S} w_{C,i}$ and $\sum_{i\in S'} w_{C,i}$ coincide with probability at most $1/(p-1)$, by the same conditioning applied to their difference. A claimed satisfied set can therefore be checked against the observed residual by one modular dot product, with error probability at most $1/(p-1)$: a randomised audit of an untrusted solver's per-clause claims that needs only $p$ somewhat larger than the clause width, not $p > 2^s$. The fingerprint verifies claims rather than supporting direct decoding; injective decoding still requires the deterministic constructions above.

# F Approximate negation

The negated-literal coefficient $-1$ may be replaced by any positive unit congruent to $-1$ modulo $p$, such as $p-1$ or more generally $p^k-1$, giving clause rows with non-negative coefficients. On the Boolean domain this is not an approximation but an identity.

**Lemma 5.** *Let $\tilde{u}_C$ agree with $u_C$ except that some negated-literal coefficients $-1$ are replaced by integers $\tilde{c} \equiv -1 \pmod p$, with $t_C$ unchanged. Then $|\tilde{u}_C^\top x - t_C|_p = |u_C^\top x - t_C|_p$ for every Boolean-labelled $x$.*

*Proof.* The two residuals differ by $\sum_{\text{substituted } i}(\tilde{c}+1)x_i$, a multiple of $p$. At the falsifying assignment every negated literal's variable is true, so $x_i = 0$ there and both residuals equal 0. At any other Boolean assignment the original residual is a unit by (R2), and adding a multiple of $p$ leaves its class modulo $p$, hence its norm, unchanged. □

The substitution is thus free at the level of the induced loss $L_\Phi(x) = \alpha n - \mathrm{sat}_\Phi(x)$, and equally free for the domination that snaps minimisers to $\{0, b_i\}^n$: since $p^k-1$ is a $p$-adic unit, $|(p^k-1)x_i|_p = |x_i|_p$, so every clause term still has norm at most 1 and every coordinatewise perturbation $c_{C,i}(x_i - x_i')$ keeps the norm it had with coefficient $-1$. The heavy unit-well weight $\alpha > \Delta$ dominates the clause rewards exactly as before. The Archimedean growth of the coefficient—$p^k-1$ is unbounded in $k$—is invisible to the $p$-adic loss, and $k=1$, the coefficient $p-1$, already suffices; larger $k$ gives the same on-domain objective.

# G Polynomial-valued false labels

The integer false labels of Section 2.3 can also be replaced by polynomials, with the loss defined by an ultrametric on a rational-function field. Work over $\mathbb{Q}(T)$, encode $z_1 =$ false by $x_1 = T$, and encode $z_2 =$ false by $x_2 = T+1$, while retaining 0 as the true value for each variable. Table 10 reproduces Table 3 with these polynomial targets.

| Row | $x_1$ | $x_2$ | Target | Signed weight | Role |
|---|---|---|---|---|---|
| $C_1$ | 1 | 0 | $T$ | $-1$ | reward $(z_1)$ |
| $C_2$ | $-1$ | 1 | $T+1$ | $-1$ | reward $(\neg z_1 \vee z_2)$ |
| $U_{1,0}$ | 1 | 0 | 0 | $+3$ | true well |
| $U_{1,T}$ | 1 | 0 | $T$ | $+3$ | false well |
| $U_{2,0}$ | 0 | 1 | 0 | $+3$ | true well |
| $U_{2,T+1}$ | 0 | 1 | $T+1$ | $+3$ | false well |

Table 10: Polynomial-target version of the two-clause dataframe. On the displayed two-point domains, each negative row vanishes exactly on the assignment that falsifies its clause.

For $C_1$, the negative residual vanishes exactly when $x_1 = T$. For $C_2$, the four values of $-x_1 + x_2$ are 0, $T + 1$, $-T$, and 1, so the target $T + 1$ isolates the falsifying assignment $(x_1, x_2) = (0, T + 1)$. Thus the two clause rows retain their zero-versus-nonzero interpretation on the displayed finite domain.

Degree defines a non-Archimedean absolute value on the rational-function field $\mathbb{Q}(T)$: for any fixed $c > 1$ and nonzero $f, g \in \mathbb{Q}[T]$,

$$\left|\frac{f}{g}\right|_\infty = c^{\deg f - \deg g}, \qquad |0|_\infty = 0.$$

Multiplicativity follows from additivity of degree under products. The inequality

$$\deg(f + g) \leq \max\{\deg f, \deg g\}$$

gives the strong triangle inequality. Hence $d_\infty(F, G) := |F - G|_\infty$ is an ultrametric.

For more than two variables, the choice of clause row does matter. Simply continuing the labels $T, T + 1, T + 2, T + 3$ does not support a direct unsigned encoding. For example, the raw-value sum for the falsifying assignment of $\neg A \vee B \vee C \vee \neg D$ is $(T+1)+(T+2) = 2T+3$, but the complementary assignment has the same sum $T+(T+3) = 2T+3$; the label sets $\{T, T+3\}$ and $\{T + 1, T + 2\}$ collide because $0 + 3 = 1 + 2$, as in any four-term arithmetic progression of constants. A direct unsigned-sum row therefore cannot distinguish these two assignments.

The clause-signed rows do not have this defect, and no per-assignment checking is needed. As with the integer labels of Section 2.3, the residual on Boolean-labelled assignments equals minus the sum of the false labels of the satisfied literals, so the row vanishes exactly at the falsifying assignment if and only if no nonempty subset of the clause's false labels sums to zero. Affine labels $f_i = T + m_i$ with distinct constants $m_i$ satisfy this condition for every clause, since a nonempty subset of size $k$ sums to $kT$ plus a constant: over $\mathbb{Q}(T)$ unconditionally, and in characteristic $p$ for clauses of width below $p$, the same width bound as the integer labels.

The degree-at-infinity norm does not automatically preserve the original clause-count loss, because different nonzero residuals can have different degrees and hence different magnitudes. The affine signed scheme, however, does preserve it: every residual at a satisfying assignment is $-(kT + \text{const})$ with $k$ between 1 and the clause width, so each satisfied clause is rewarded by the same magnitude $c^1$, and on Boolean-labelled assignments the loss again decreases by one unit of reward per satisfied clause. Uniform magnitude and injective residual values can even be combined: with $m_i = 2^i$ the residual determines exactly which literals of the clause are satisfied, the cardinality from the leading coefficient and the subset from the constant term. We have not developed an application of these labelled residuals and record the construction as a direction for future research.

## Funding

This research was supported by an Australian Government Research Training Program Scholarship.

## Compliance with ethical standards

This article does not contain any studies with human participants or animals performed by the author.

## Conflict of interest

The author declares that he has no conflict of interest.

## Author contributions

This is a single-author paper. The author is responsible for the conceptual development, proofs, implementation, and writing.

## Code and data availability

The Python implementation used for the heuristic experiments is publicly available at:

`https://github.com/solresol/sudoku-padic-regression`
Dataset archive: Hugging Face dataset
Experiment-code revision: `ba49a8b2e51670f66db624d814d3f31678881b93`
Main scripts: `code/padic_sudoku_regression.py`, `code/run_experiments.py`
Powers-of-two script: `archive/scripts/run_experiments.py`
Raw E1 data: `archive/results/e1_prime_sweep.csv`
Raw E2 data: `archive/results/e2_heuristic_comparison.csv`

The submitted source package includes the experiment CSV files, puzzle seeds, trace puzzle and solution files, and figure source used for Tables 8 and 9 and Figure 6. The raw-data revision above generated the archived CSV rows; the submitted driver corrects the summary formatter so that half-integer medians are not truncated. The main experiment outputs were produced with Python 3.11.6 and Matplotlib 3.9.0 on an Apple M1 system. Timing columns are retained as descriptive provenance only.

### Interactive demonstration

A browser-based demonstration of the construction is shown in Figure 3 and available at

`https://padic-logic.symmachus.org`
`https://padic-logic.symmachus.org/#sudoku`

The Sudoku and Boolean-CSP compilers, residual dataframes, objective evaluations, and representative searches run client-side. Their source is maintained separately at `https://github.com/solresol/padic-logic`, with the paper-linked application snapshot pinned at revision `a8130ee`; the deployed values shown in Figure 3 were checked on 13 July 2026. The optional natural-language schema extractor requires a browser exposing the on-device Prompt API and is therefore not part of the browser-independent reproducibility claim.

## References


[1] G. Baker, S. McCallum, and D. Pattinson. Linear regression in $p$-adic metric spaces. *p-Adic Numbers, Ultrametric Analysis and Applications* 17(4), 333–347 (2025). `https://doi.org/10.1134/S2070046625040016`

[2] G. Baker. Expressivity and NP-hardness of $p$-adic linear regression. *p-Adic Numbers, Ultrametric Analysis and Applications* 18(2), 210–216 (2026). `https://doi.org/10.1134/S2070046626020081`

[3] T. Schiex, H. Fargier, and G. Verfaillie. Valued constraint satisfaction problems: hard and easy problems. In *Proceedings of IJCAI-95*, 631–639 (1995). `https://www.ijcai.org/Proceedings/95-1/Papers/083.pdf`

[4] J.-C. Régin. A filtering algorithm for constraints of difference in CSPs. In *Proceedings of AAAI-94*, 362–367 (1994). `https://aaai.org/Papers/AAAI/1994/AAAI94-055.pdf`

[5] J. A. Ellis-Monaghan and I. Moffatt. A note on recognizing an old friend in a new place: list coloring and the zero-temperature Potts model. *Annales de l'Institut Henri Poincaré D* 1(4), 429–442 (2014). https://doi.org/10.4171/AIHPD/12

[6] A. Lucas. Ising formulations of many NP problems. *Frontiers in Physics* 2, 5 (2014). https://doi.org/10.3389/fphy.2014.00005

[7] S. Minton, M. D. Johnston, A. B. Philips, and P. Laird. Solving large-scale constraint-satisfaction and scheduling problems using a heuristic repair method. In *Proceedings of AAAI-90*, 17–24 (1990). https://cdn.aaai.org/AAAI/1990/AAAI90-003.pdf

[8] R. Lewis. Metaheuristics can solve sudoku puzzles. *Journal of Heuristics* 13(4), 387–401 (2007). https://doi.org/10.1007/s10732-007-9012-8

[9] Donald E. Knuth. *Dancing Links. Millennial Perspectives in Computer Science*, 187–214, 2000. Also available as arXiv:cs/0011047. https://doi.org/10.48550/arXiv.cs/0011047

[10] Helmut Simonis. *Sudoku as a Constraint Problem.* In *Modelling and Reformulating Constraint Satisfaction Problems*, Fourth International Workshop, Sitges (Barcelona), Spain, October 1, 2005. https://citeseerx.ist.psu.edu/document?doi=4f069d85116ab6b4c4e6dd5f4776ad7a6170faaf

[11] Inês Lynce and Joël Ouaknine. *Sudoku as a SAT Problem.* In *Proceedings of the Ninth International Symposium on Artificial Intelligence and Mathematics (ISAIM 2006)*, 2006. https://people.mpi-sws.org/~joel/publications/sudoku05abs.html

[12] T. Yato and T. Seta. Complexity and completeness of finding another solution and its application to puzzles. *IEICE Transactions on Fundamentals*, 86-A(5):1052–1060, 2003.

[13] A. P. Zubarev. $p$-Adic polynomial regression as alternative to neural network for approximating $p$-adic functions of many variables. *p-Adic Numbers, Ultrametric Analysis and Applications* 17(4), 413–420 (2025). https://doi.org/10.1134/S2070046625040065

[14] A. F. T. Martins. Learning with the $p$-adics. arXiv:2512.22692 (2025). https://arxiv.org/abs/2512.22692

[15] T. Mihara. $p$-adic linear regression for random sampling with digitwise noise. arXiv:2604.13137 (2026). https://arxiv.org/abs/2604.13137

[16] T. Mihara. Generalisation of Baker's forcing method to arbitrary prime and NP-hardness of several $p$-adic optimisations. arXiv:2607.06092 (2026). https://arxiv.org/abs/2607.06092